ICPE with GPT-2 networks: architecture, training and inference (algo/ implementation, K = N = 4, fixed confidence)

1 Network architecture

Same design for Q_θ and I_ϕ, separate weights. 4 copies in memory:

Q_θ, target $\bar{Q}_\theta$, I_ϕ, target $\bar{I}_\phi$ (icpe_setup.py, reset_networks)

History D_t: 4 rows, row = one-hot arm (4) + reward (1)

row 1 [1 0 0 0 | .55] · **row 2** [0 1 0 0 | .72] · **row 3** [0 0 0 0 | 0] empty · **row 4** [0 0 0 0 | 0] empty

ICPE embed: Linear 5 → 256, then GELU

(B, 4, 5) → (B, 4, 256) icpe_setup.py:199, 216

GPT2Model (Hugging Face, random weights, NOT pretrained)

inputs_embeds=h skips the word table · n_positions=4 · icpe_setup.py:197, 218

+ position embedding (wpe)

tells the model which row is 1st, 2nd, 3rd, 4th

Transformer block × 3 (n_layer = 3)

- **LayerNorm**
- **Masked self-attention: 8 heads × 32 dims**
  rows exchange info, row i sees rows 1..i only · + residual
- **LayerNorm**
- **MLP 256 → 1024 → 256 (GELU), each row alone · + residual**

Final LayerNorm → last_hidden_state (B, 4, 256)

Keep only the last row: h[:, -1, :]

(B, 256): row 4 has attended to rows 1–4

Q_θ head: Linear 256 → 5

critic / "Chooser"

arm0 arm1 arm2 arm3 STOP

0.70 0.65 0.73 0.75 | 0.37 → pull arm 3

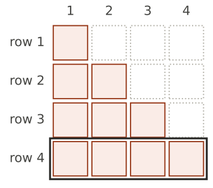
I_ϕ head: Linear 256 → 4

log-softmax / "Guesser"

P(best): arm0 arm1 arm2 arm3

0.00 0.47 0.26 0.27 → guess arm 1

Causal attention mask (who can look at whom)



row 4 sees all rows → only its output goes to the heads.

No padding mask: empty rows are zeros + position; the net learns that a zero row means "not pulled yet".

2 Training (Algorithm 1, lines 3–12) icpe_training.py: train()

Sample 40 new bandits

means ~ U[0,1], std 0 · line 4

Q_θ chooses a_t

argmax Q (random arms in warm-up)

Bandit step

arm: add row, reward = mean · STOP: end

Replay buffer B (5M slots)

store (D_t, a_t, D_{t+1}, done, H^*)

↻ next pull (t = 1..4) until all 40 are done

Update I_ϕ (panel 4)

once buffer > 2,560 memories

Update Q_θ (panel 3)

once buffer > 7,680 memories

Target copies

$\bar{Q}_\theta$: 1% per step · $\bar{I}_\phi$: copy every 8

After the episode: update λ = price per pull (eq. 11, line 9)

err = (1−δ) − accuracy of I_ϕ on the 40 final histories; $\lambda \leftarrow \lambda$ − AdamW(err), clamp to [0, 1/N]

accuracy too low → λ down → explore longer · accuracy too high → λ up → stop sooner

↻ 100,000 steps × 40 bandits = 4 million training episodes

one gradient step for each network after EVERY pull, each on its own random batch of 256

3 Q-learning update of Q_θ icpe_q_learning.py: train_rl_nets()

Batch of 256 random memories from B

D_t, a_t, D_{t+1}, done, H^*

$Q_\theta(D_t)$ — online · Q of a_t

$Q_\theta(D_{t+1})$ — online · picks a'

$\bar{Q}_\theta(D_{t+1})$ — target · Q of a'

$\bar{I}_\phi(D_{t+1})$ — r = P(H^*)

Target y = −λ(1−done) + done·r + 0.999·(1−done)·$\bar{Q}_\theta(D_{t+1}$, a')

no gradients · Double DQN (online picks, target scores)

Loss = MSE($Q_\theta(D_t, a_t)$, y) + MSE($Q_\theta(D_{t+1}$, STOP) , r)

2nd term "pretend to stop": STOP value = confidence now · only where $a_t \neq$ STOP · fixed confidence only

Backprop into Q_θ only · clip grad 2 · AdamW lr 1e-5

then $\bar{Q}_\theta \leftarrow 0.01 \cdot Q_\theta + 0.99 \cdot \bar{Q}_\theta$ (Polyak)

Key idea: the reward is the Guesser's confidence in the true best arm, NOT the bandit reward.

So Q_θ learns to collect the evidence that makes I_ϕ right, paying λ per pull.

4 I_ϕ update (classification, eq. 7) train_estimation_net()

256 memories

D_{t+1} and H^*

$I_\phi(D_{t+1})$

4 log-probabilities

pick column H^*

log $I_\phi(H^*|D_{t+1})$

Loss = −mean

cross-entropy

Backprop into I_ϕ · clip grad 2 · AdamW lr 1e-5 · every 8 updates: $\bar{I}_\phi \leftarrow$ exact copy of I_ϕ

I_ϕ is used for: (1) reward r via $\bar{I}_\phi$ (2) accuracy for the λ update (3) the final answer $\hat{H}$

5 Inference (Algorithm 1, lines 13–15) icpe_inference.py

A brand-new bandit (never seen in training)

Q_θ: argmax, always greedy (no randomness)

STOP chosen, or all 4 pulls used?

no: pull the arm, add the row

yes

I_ϕ reads the final history

answer $\hat{H}$ = argmax $I_\phi(\cdot|D_\tau)$

Record the result

τ = number of real pulls · correct = ($\hat{H} = H^*$)

Your results

300,000 test bandits per δ

δ = 0.1: avg τ 3.56, acc 0.978

δ = 0.01: avg τ 3.72, acc 0.989

$\delta \leq$ 0.001: avg $\tau \approx$ 4.0

6 Key settings (icpe_setup.py CONFIG)

K arms / N max pulls	4 / 4
Q_θ outputs	4 arms + STOP = 5
I_ϕ outputs	4 (one per arm)
GPT-2 size	width 256, 3 layers, 8 heads, 4 positions, ~2.37M used params
Batch / buffer	256 / 5,000,000 memories
Learning rate	1e-5 (both nets) · λ: 1e-4
γ / Polyak	0.999 / 0.01
Warm-up	random arms until 7,680 memories
Training	100,000 steps × 40 bandits (seed 1)
λ start / range	0.125 / [0, 0.25] target accuracy 1 − δ

Q_θ "Chooser" (critic) · I_ϕ "Guesser" (inference) · GPT-2 internals · λ / results · data and plumbing